MACHINE LEARNING WITH PYTHON PROJECT REPORT

(Project Semester August-December 2026)

HEART DISEASE PREDICTION USING

MACHINE LEARNING MODELS

Submitted by

Aakash Shaji

Registration No. 12611114

Programme and Section: M. Tech Data Science & Analytics (K3P26DS)

Course Code: CSE570



Under the Guidance of Pardeep Kumar

School of Computing and AI

Lovely Professional University

TABLE OF CONTENTS

1. INTRODUCTION 5
1.1 Introduction 5
1.2 Problem Statement 5
1.3 Objectives 5
1.4 Scope of the Project 6
1.5 Significance of the Project 6
2. DATASET DESCRIPTION 7
2.1 Dataset Source 7
2.2 Important Attributes 7
2.3 Dataset Usage in the Project 8
2.4 Suitability of the Dataset 8
2.5 Dataset Quality and Target Distribution 8
3. DATA PREPROCESSING 9
3.1 Introduction 9
3.2 Data Loading and Inspection 9
3.3 Handling Missing Values 9
3.4 Data Cleaning 10
3.5 Feature Selection 10
3.6 Feature Engineering and Target Variable Generation 11
3.7 Encoding and Scaling 11
4. EXPLORATORY DATA ANALYSIS 12
4.1 Introduction 12
4.2 Dataset Overview 12
4.3 Class Distribution 12
4.4 Heart Disease by Sex 13
4.5 Average Values in Each Group 13
4.6 Age and Maximum Heart Rate 14
4.7 Thalassemia, Cholesterol, Blood Pressure and Blood Sugar 15
4.8 Correlation Between Numeric Features 16
4.9 Summary of the Findings 17
5. METHODOLOGY 18
5.1 Introduction 18
5.2 Overall System Workflow 18
5.3 Machine Learning Workflow 18
5.4 Why a Pipeline Is Used 19

5.5 Choosing k for K-Nearest Neighbours 19
5.6 Prediction for a New Patient 19
6. MODEL DEVELOPMENT 20
6.1 Introduction 20
6.2 Feature Selection 20
6.3 Feature Extraction and Transformation 20
6.4 Training and Testing Datasets 20
6.5 Classification Algorithms 20
6.6 Classifier Training and Predictions 22
6.7 Parameter Setup 22
7. RESULTS AND MODEL COMPARISON 23
7.1 Introduction 23
7.2 Evaluation Metrics 23
7.3 Model Performance Comparison 23
7.4 Confusion Matrix Analysis 24
7.5 Classification Report 26
7.6 Overall Result Analysis 27
7.7 Prediction for a New Patient 27
8. CONCLUSION AND LIMITATIONS 28
8.1 Conclusion 28
8.2 Limitations 28
8.3 Future Enhancements 28
8.4 Final Remarks 29
9. CODE 34
10. REFERENCES 34

ABSTRACT

Heart disease is one of the leading causes of death worldwide, and an early diagnosis can change the outcome for a patient considerably. In this project I build and compare several machine learning classifiers that predict whether a patient has heart disease from thirteen routine clinical measurements. The data comes from the Cleveland heart disease database, which has 303 patient records. Before modelling, the data was checked for duplicate rows and for impossible category codes. One duplicate record was removed, and invalid category codes in two columns (num_major_vessels and thalassemia) were treated as missing values and filled later inside the model pipeline so that no information from the test set leaked into training.

Exploratory analysis was carried out with pandas, matplotlib and seaborn to study the class balance, the effect of sex and age, and the relationship between the numeric measurements. Five classifiers were trained on an 80:20 stratified split: Gaussian Naive Bayes, a depth-limited Decision Tree, K-Nearest Neighbours with three different values of k, and Logistic Regression. The models were compared using test accuracy, false negative rate, five-fold cross-validated accuracy and ROC AUC. All models reached close to 88.5% accuracy on the small test set, but Logistic Regression gave the best cross-validated accuracy (85.77%), the best ROC AUC (0.953) and, together with the Decision Tree, the lowest false negative rate (17.86%). The work is meant for learning and is not a replacement for a clinical diagnosis.

Keywords: heart disease, classification, logistic regression, K-nearest neighbours, cross-validation, scikit-learn

1. INTRODUCTION

1.1 Introduction

Cardiovascular diseases cause more deaths every year than any other group of illnesses. A large part of the problem is that many patients are diagnosed late, after symptoms have already become serious. Doctors usually decide using a combination of tests such as blood pressure, cholesterol level, an ECG and a stress test, and the interpretation of these numbers takes experience.

Machine learning offers a way to learn such patterns from past patient records. If a model can pick up the combination of values that usually goes with a diagnosed heart condition, it can act as a second opinion or a screening aid. In this project I use Python and scikit-learn to build such a model on a well-known public dataset, compare different algorithms, and check how reliable the results are.

1.2 Problem Statement

Given a patient's age, sex, type of chest pain, blood pressure, cholesterol, ECG result, heart rate during exercise and a few other test results, the task is to predict whether the patient has heart disease or not. This is a binary classification problem. The difficulty is that the dataset is small, the features are a mix of numbers and category codes, and a wrong prediction is not equally costly in both directions: calling a sick patient healthy is a much more serious mistake than the opposite.

1.3 Objectives

The main objectives of the project are:

- To load, inspect and clean the Cleveland heart disease dataset, including hidden problems such as duplicates and invalid category codes.
- To carry out exploratory data analysis with tables and plots and to understand which measurements differ between patients with and without heart disease.
- To build a preprocessing pipeline that handles missing values, scaling and encoding without leaking test data.
- To train Naive Bayes, Decision Tree, K-Nearest Neighbours and Logistic Regression classifiers.
- To select the value of k for KNN using cross-validation on the training data instead of the test data.

- To compare the models using accuracy, false negative rate, cross-validated accuracy, confusion matrices and ROC AUC.
- To use the best model to predict the risk for a new patient record.

1.4 Scope of the Project

The project covers one dataset, the Cleveland subset of the UCI Heart Disease data, and one prediction target: presence or absence of heart disease. The work includes data cleaning, exploratory analysis, model training and evaluation, and a prediction example. It does not include real-time data, medical images or deployment as a web application. The project is made for academic learning, and its predictions should never be used to take a real medical decision.

1.5 Significance of the Project

The project shows the full path of a small but realistic machine learning task: finding problems in raw data, preparing it properly, comparing several models fairly and reading the results carefully. Two points received special attention. The first is data leakage, which is avoided by putting all preprocessing inside a pipeline. The second is the false negative rate, which is reported next to accuracy because accuracy alone hides the cost of missing a sick patient. These habits are useful in any machine learning project, not only in healthcare.

2. DATASET DESCRIPTION

2.1 Dataset Source

The dataset used here is the Cleveland Heart Disease dataset. It was collected at the Cleveland Clinic Foundation and is part of the Heart Disease collection of the UCI Machine Learning Repository, which also contains data from Hungary, Switzerland and Long Beach. The Cleveland subset is the one most researchers use because it has almost no missing values. A copy of it is circulated on Kaggle as a single CSV file, and that file (heart.csv) is the one I worked with.

Dataset details:

Item	Details
Dataset name	Cleveland Heart Disease dataset
Original source	UCI Machine Learning Repository (Cleveland Clinic Foundation)
File used	heart.csv
Format	CSV (comma-separated values)
Number of records	303 (302 after removing one duplicate row)
Number of attributes	14 (13 input features and 1 target)
Task type	Binary classification

2.2 Important Attributes

Each row describes one patient. The columns are a mix of measurements and coded categories. In the notebook I renamed the short original column names into readable ones, and Table 2.1 lists both together with the meaning of the codes.

Table 2.1: Attributes of the dataset

Original name	Name used	Description and values
age	age	Age of the patient in years
sex	sex	0 = female, 1 = male
cp	chest_pain_type	0 = asymptomatic, 1 = atypical angina, 2 = non-anginal pain, 3 = typical angina
trestbps	resting_blood_pressure	Resting blood pressure in mm Hg
chol	cholesterol	Serum cholesterol in mg/dl
fbs	fasting_blood_sugar	0 = at most 120 mg/dl, 1 = above 120 mg/dl
restecg	rest_ecg	0 = left ventricular hypertrophy, 1 = normal, 2 = ST-T wave abnormality
thalach	max_heart_rate_achieved	Highest heart rate reached during the exercise test
exang	exercise_induced_angina	0 = no, 1 = yes
oldpeak	st_depression	ST depression caused by exercise compared with rest
slope	st_slope	0 = downsloping, 1 = flat, 2 = upsloping
ca	num_major_vessels	Number of major vessels (0 to 3) coloured by fluoroscopy

thal	thalassemia	1 = fixed defect, 2 = normal, 3 = reversible defect
target	target	Final label. After my correction: 0 = no disease, 1 = disease

2.3 Dataset Usage in the Project

The CSV file is read into a pandas DataFrame. I first examine its structure, data types, summary statistics and quality problems. Then I create two copies of the data. One copy keeps the numeric codes and is used for training the models. The second copy, called eda in the notebook, replaces the codes with readable words such as “male”, “typical angina” or “flat”, and it is used only for plots and tables, so that the graphs can be read without a code book.

All 13 input features are used for modelling. The six numeric ones (age, resting blood pressure, cholesterol, maximum heart rate, ST depression and number of major vessels) are scaled, while the seven categorical ones are one-hot encoded..

2.4 Suitability of the Dataset

The dataset suits this project for a few reasons. It is a real clinical dataset that has been used in many published studies, so the results can be discussed against a known background. Its size is small enough to train any model in a few seconds, which makes repeated cross-validation easy. It has both numeric and categorical variables, so it needs a proper preprocessing step. It also has a clearly defined binary target. The small size is also a weakness, and I come back to it in the limitations.

2.5 Dataset Quality and Target Distribution

The dataset does not contain empty cells when it is loaded, since info() reports 303 non-null values in every column. This can be misleading, however. Two columns hold coded values, and some codes fall outside the documented range: five rows have num_major_vessels equal to 4, although the valid values are 0 to 3, and two rows have thalassemia equal to 0, although the valid codes are 1 to 3. These are effectively hidden missing values. In addition, one row was an exact duplicate of another one.

After the duplicate was dropped, 302 patient s remained: 164 without heart disease (54.3%) and 138 with heart disease (45.7%). The classes are therefore fairly balanced, and no resampling was needed. By sex, 96 patients are female (31.79%) and 206 are male (68.21%).

3. DATA PREPROCESSING

3.1 Introduction

Raw data is rarely ready for a model. In this project the preprocessing work has two parts. The first part is cleaning that is done once on the whole table: removing the duplicate row, marking invalid codes as missing, renaming columns and fixing the target label. The second part is done inside a scikit-learn Pipeline, so that it is learned from the training data only: filling missing values, scaling the numeric columns and encoding the categorical columns. Keeping these two parts apart is what protects the evaluation from data leakage.

3.2 Data Loading and Inspection

The file heart.csv is loaded with pandas read_csv(). The shape of the table is 303 rows and 14 columns. I looked at the data with head(), sample(), describe(), info() and shape. All columns are stored as integers except st_depression (oldpeak), which is a float. There are no empty cells at this stage.

The summary statistics of the raw 303 records are given in Table 3.1. The age range is 29 to 77 years and the average is about 54. Cholesterol goes up to 564 mg/dl, which is far above the typical range and is the most extreme value in the table, but it is a possible measurement, so I kept it.

Table 3.1: Summary of the numeric features (raw data, 303 records)

Feature	Mean	Std. dev.	Min	Max
age	54.37	9.08	29	77
resting_blood_pressure	131.62	17.54	94	200
cholesterol	246.26	51.83	126	564
max_heart_rate_achieved	149.65	22.91	71	202
st_depression	1.04	1.16	0.0	6.2

3.3 Handling Missing Values

Since info() showed no empty cells, I checked the value counts of the coded columns. The counts were:

- num_major_vessels: 0 appears 175 times, 1 appears 65 times, 2 appears 38 times, 3 appears 20 times and 4 appears 5 times.
- thalassemia: 0 appears 2 times, 1 appears 18 times, 2 appears 166 times and 3 appears 117 times.

The value 4 for the number of vessels and the value 0 for thalassemia are not valid codes, so I replaced them with NaN. After removing the duplicate row, 4 missing values remain in num_major_vessels and 2 in thalassemia (Table 3.2).

Table 3.2: Hidden missing values found in the data

Column	Invalid value	Rows (raw data)	Rows (after duplicate removal)
num_major_vessels	4	5	4
thalassemia	0	2	2

These few gaps are not filled at this stage. They are filled inside the pipeline, using the median for numeric columns and the most frequent value for categorical columns, and the fill values are calculated from the training set only. If the fill values were computed on the full data before splitting, a small amount of information from the test patients would pass into training, which is the leakage I wanted to avoid.

3.4 Data Cleaning

Three cleaning steps were carried out.

1. **Duplicate removal.** One row was an exact copy of another. If it had stayed, the same patient could land in both the training set and the test set and make the test score look better than it should. After drop_duplicates() the table has 302 rows.
2. **Invalid codes.** The impossible values described above were converted to missing values.
3. **Renaming columns.** The short original names such as cp, thalach and oldpeak were renamed to readable names (see Table 2.1). A separate copy of the table with text labels was prepared for plotting, using the map() function, so the numeric data used for training stays untouched.

3.5 Feature Selection

Only thirteen input features exist, and all of them are clinically meaningful, so I kept all of them. I did not drop any column by hand. What I did was to divide the features into two groups, because the two groups need different treatment (Table 3.3).

Table 3.3: Numeric and categorical features

Group	Features	Treatment
Numeric (6)	age, resting_blood_pressure, cholesterol, max_heart_rate_achieved, st_depression, num_major_vessels	Median imputation, then standard scaling

Categorical (7)	sex, chest_pain_type, fasting_blood_sugar, rest_ecg, exercise_induced_angina, st_slope, thalassemia	Most-frequent imputation, then one-hot encoding

The number of major vessels is treated as a number because it is a count from 0 to 3 and the order is meaningful. The categorical columns, on the other hand, hold labels. For example, chest-pain code 3 is not "three times" code 1, so these columns should not be fed to a model as plain numbers.

3.6 Feature Engineering and Target Variable Generation

There was little need for new features, since the data is already a set of engineered clinical measurements. The one important transformation was on the target. As explained in Section 2.5, the original label seemed to be reversed, so I set target = 1 − target. After this change the class counts are 164 patients labelled 0 (no disease) and 138 labelled 1 (disease). The notebook keeps a switch called FLIP_TARGET, so the original meaning can be restored with one change if needed.

3.7 Encoding and Scaling

Models like K-Nearest Neighbours and Logistic Regression are sensitive to the size of the numbers. Cholesterol is in the hundreds, whereas ST depression is mostly below 3, so without scaling the large values would dominate the distance calculation in KNN. StandardScaler solves this by giving each numeric column a mean of 0 and a standard deviation of 1. For categorical columns, OneHotEncoder creates a separate 0/1 column for every category, with handle_unknown set to "ignore" so that an unseen category does not break prediction.

Both branches are joined with a ColumnTransformer and attached in front of every classifier through a Pipeline.

4. EXPLORATORY DATA ANALYSIS

4.1 Introduction

Exploratory data analysis (EDA) is done to understand the data before any model is trained. In this chapter I look at the class balance, the role of sex and age, the average values of each feature in the two groups, a few scatter plots and the correlation between the numeric features. The plots were made with matplotlib and seaborn from the readable copy of the data.

4.2 Dataset Overview

After cleaning, the dataset has the following overall picture.

Table 4.1: Overview of the cleaned dataset

Measure	Value
Total patients	302
Input features	13
Patients without heart disease	164 (54.30%)
Patients with heart disease	138 (45.70%)
Female patients	96 (31.79%)
Male patients	206 (68.21%)

4.3 Class Distribution

Figure 4.1 shows the number of patients in each class. The two bars are close in height, so the dataset is reasonably balanced. This is good news for the modelling, because accuracy is not misleading here as it would be in a dataset where 95% of the cases belong to one class.

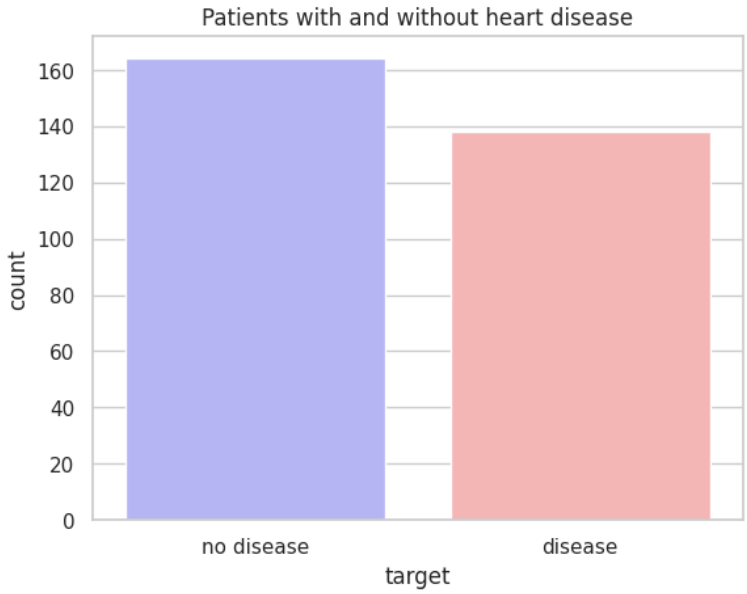


Figure 4.1: Patients with and without heart disease

4.4 Heart Disease by Sex

Male patients make up about two thirds of the data. The disease rate also differs a lot between the sexes: roughly one in four female patients has heart disease, compared with a little over half of the male patients (Figure 4.2). Figure 4.3 shows the same difference as counts. The result matches the usual medical picture, and it was also the reason why I decided that the target label needed to be flipped.

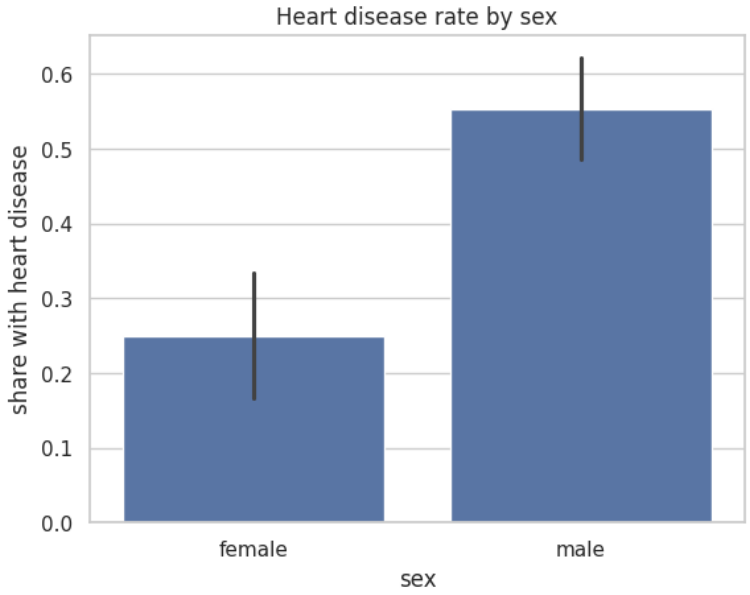


Figure 4.2: Share of patients with heart disease, by sex

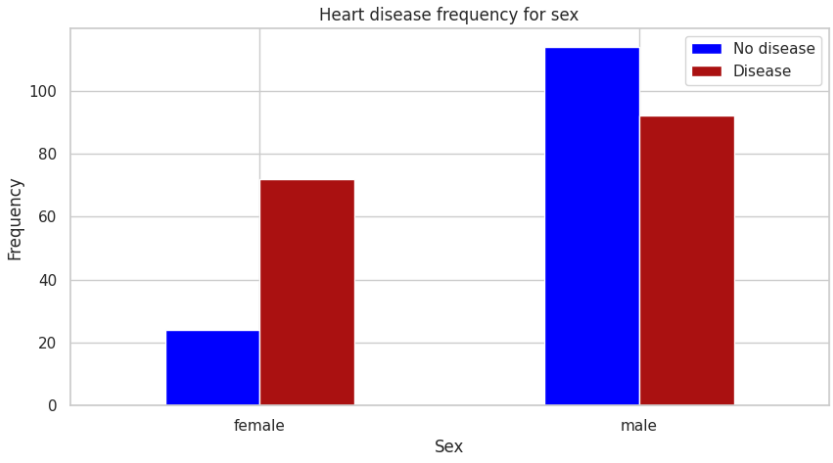


Figure 4.3: Heart disease frequency for males and females

4.5 Average Values in Each Group

Table 4.2 gives the mean of each feature separately for patients with and without disease. Some differences are large. Patients with heart disease reached a much lower maximum heart rate (139.10 against 158.38), had far more exercise-induced angina (0.55 against 0.14 as a share of patients), a higher ST depression (1.59 against 0.59) and more major vessels affected (1.15

against 0.27). They were also older on average (56.60 years against 52.59). On the other hand, fasting blood sugar (0.16 against 0.14) and cholesterol (251.09 against 242.64) differ only a little, so these two are probably weak signals on their own.

Table 4.2: Mean value of each feature by class

Feature	No disease (0)	Disease (1)
age	52.59	56.60
sex (share of males)	0.56	0.83
chest_pain_type (code)	1.37	0.48
resting_blood_pressure	129.25	134.40
cholesterol	242.64	251.09
fasting_blood_sugar (share)	0.14	0.16
rest_ecg (code)	0.59	0.45
max_heart_rate_achieved	158.38	139.10
exercise_induced_angina (share)	0.14	0.55
st_depression	0.59	1.59
st_slope (code)	1.59	1.17
num_major_vessels	0.27	1.15
thalassemia (code)	2.13	2.56

The chest-pain code needs a careful reading. Its average is lower for the diseased group because code 0 means “asymptomatic”. In other words, many patients with a diagnosed heart condition reported no typical chest pain at all, which is one of the less intuitive findings in this dataset.

4.6 Age and Maximum Heart Rate

Figure 4.4 shows the number of patients with and without disease for each age. Disease cases are spread across many ages, so age alone does not separate the two groups, although the average age is higher for the diseased group. Figure 4.5 plots age against the maximum heart rate achieved. The maximum heart rate drops as age increases, which agrees with the correlation of −0.40 reported later, and at similar ages the patients with heart disease tend to sit lower on the plot.

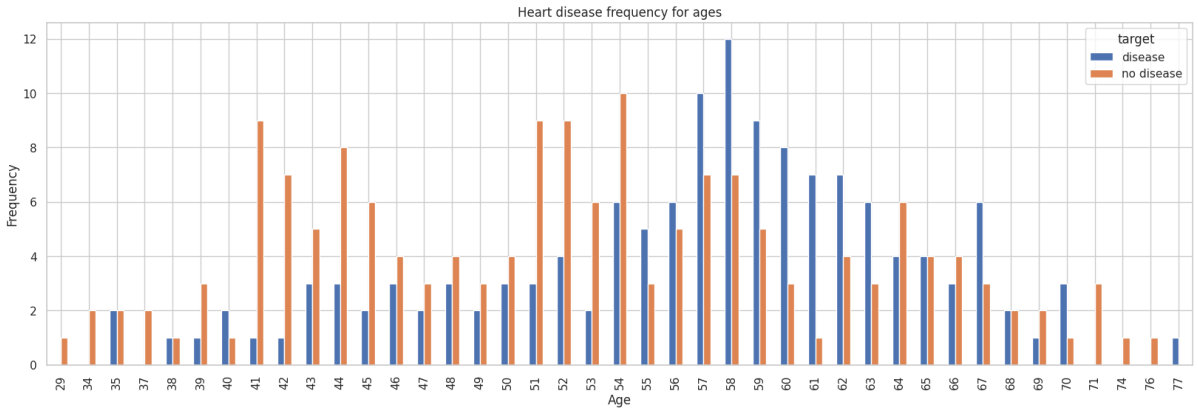


Figure 4.4: Heart disease frequency for each age

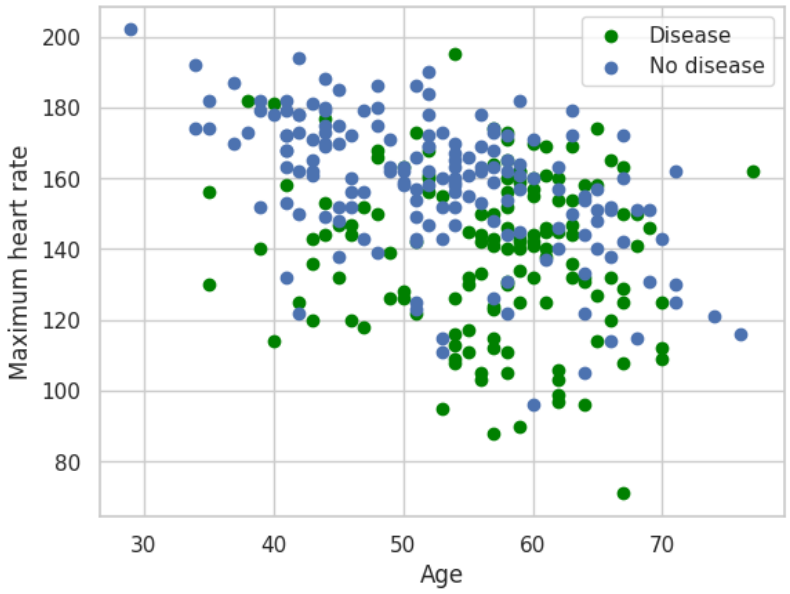


Figure 4.5: Age versus maximum heart rate

4.7 Thalassemia, Cholesterol, Blood Pressure and Blood Sugar

Thalassemia is a coded column with only three valid categories, so scatter plots against it appear as horizontal or vertical bands. Figures 4.6 and 4.7 compare it with cholesterol and with resting blood pressure, with the colour showing the class. The points overlap heavily, which means cholesterol and blood pressure do not split the classes by themselves. Figure 4.8 compares the disease counts for the two fasting blood sugar groups, and the pattern is similar in both groups, which fits the small difference seen in Table 4.2.

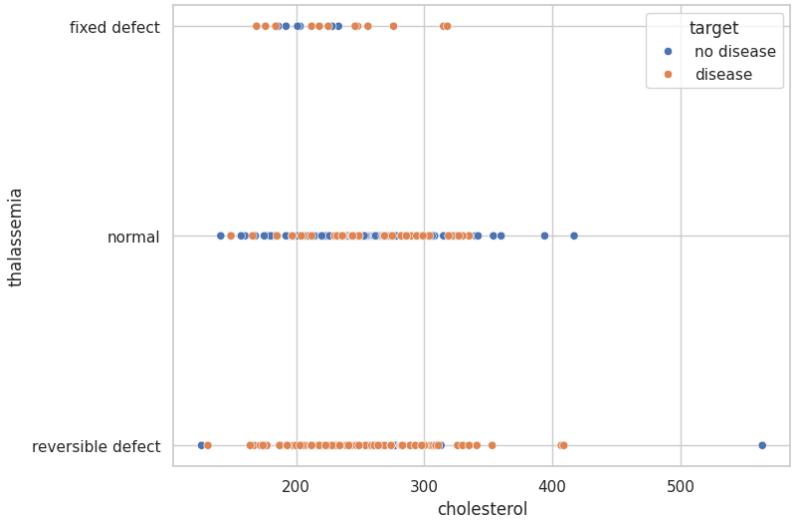


Figure 4.6: Thalassemia versus cholesterol

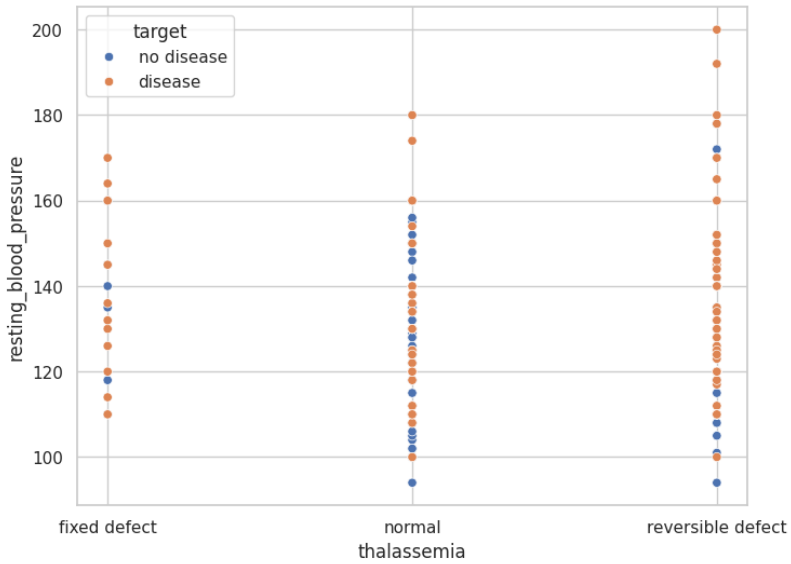


Figure 4.7: Thalassemia versus resting blood pressure

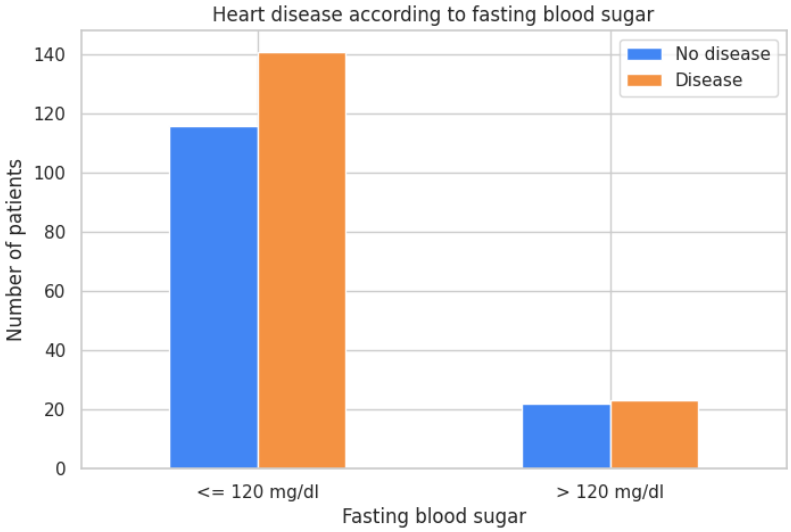


Figure 4.8: Heart disease according to fasting blood sugar

4.8 Correlation Between Numeric Features

The correlation heatmap in Figure 4.9 covers the six numeric features. None of the pairs is very strongly correlated, which means there is no serious redundancy between these features. The strongest relation is the negative one between age and maximum heart rate (−0.40). Maximum heart rate is also negatively related to ST depression (−0.34) and to the number of major vessels (−0.26). Age has a positive correlation with the number of major vessels (0.36). Cholesterol has almost no linear relation with the other numeric features.

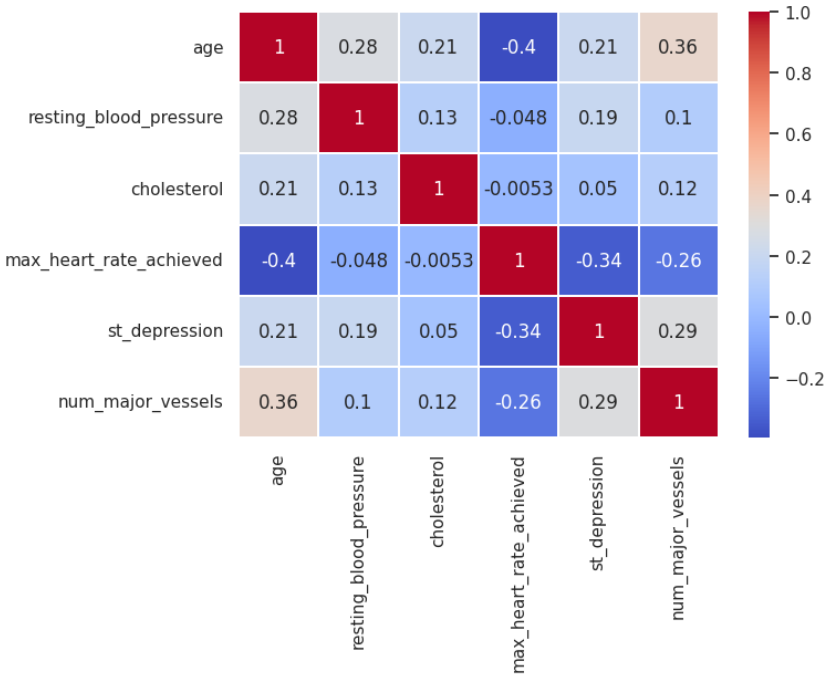


Figure 4.9: Correlation heatmap of the numeric features

4.9 Summary of the Findings

- The classes are fairly balanced (54.3% and 45.7%).
- Males, older patients and patients with exercise-induced angina show a higher rate of disease.
- Low maximum heart rate, high ST depression and more affected vessels go along with disease.
- Cholesterol and fasting blood sugar show only small differences between the classes.
- The numeric features are only moderately correlated, so all of them can be kept.

5. METHODOLOGY

5.1 Introduction

The method follows a step-by-step plan from loading the file to predicting for a new patient. All work was written in Python in a Jupyter notebook, using pandas and NumPy for data handling, matplotlib and seaborn for plots, and scikit-learn for preprocessing, models and evaluation. A fixed random seed (123) is set at the start so the results are the same on every run.

5.2 Overall System Workflow

1. **Data loading:** read heart.csv into a pandas DataFrame.
2. **Inspection:** check shape, data types, statistics, duplicates and unusual values.
3. **Cleaning:** drop the duplicate row, convert invalid codes to missing values, rename the columns and correct the direction of the target.
4. **EDA:** study class balance, sex, age and correlations with plots.
5. **Splitting:** divide the data into 80% training and 20% test sets with stratification.
6. **Preprocessing pipeline:** impute, scale and encode inside a Pipeline.
7. **Model training:** train Naive Bayes, Decision Tree, KNN (three values of k) and Logistic Regression.
8. **Evaluation:** compare accuracy, false negative rate, cross-validated accuracy, confusion matrices and ROC AUC.
9. **Prediction:** use the selected model to score a new patient record.

5.3 Machine Learning Workflow

The features X are all columns except target, and the label y is the target column. The data is split once, with test_size = 0.20, random_state = 123 and stratify = y. The stratified split keeps the share of diseased patients about the same in both parts. This gives 241 training patients and 61 test patients. The test set holds 33 patients without disease and 28 with disease.

Every model is wrapped in the same pipeline, so it gets exactly the same preprocessing. For every model I report the following:

- **Test accuracy** on the 61 hidden test patients.
- **False negative rate (FNR):** the percentage of sick patients that the model wrongly calls healthy.

- **Cross-validated accuracy:** the mean and standard deviation of accuracy over five stratified folds on the full data.
- **ROC AUC** on the test set.

The reason for including cross-validation is the small test set. With only 61 patients, a single patient changes the accuracy by about 1.6 percentage points, so one lucky or unlucky split can mislead. Averaging over five different splits gives a steadier number.

5.4 Why a Pipeline Is Used

A Pipeline chains the preprocessing and the classifier into a single object. When cross-validation is run, the imputer, scaler and encoder are fitted again inside each training fold, and the validation fold is only transformed. Therefore the validation data never influences the medians, means or categories. A second advantage is convenience: a trained pipeline accepts raw patient values and does the scaling and encoding by itself, which makes the final prediction step simple.

5.5 Choosing k for K-Nearest Neighbours

The number of neighbours k is a setting that has to be chosen. A simple way is to try many values and pick the one with the best test score, but this uses the test set for tuning and makes the final test result too optimistic. I avoided this by trying all k from 1 to 25 with five-fold cross-validation on the training data only. The best value found in this way was k = 17. For comparison I also kept k = 7 and k = 4 as extra runs.

5.6 Prediction for a New Patient

The last step uses the trained Logistic Regression pipeline on a new record given as raw numbers. The model returns a probability of heart disease, and a probability of 0.5 or more is labelled as “disease”. This part of the notebook only demonstrates how the model could be used, and it is not a validated clinical tool.

6. MODEL DEVELOPMENT

6.1 Introduction

This chapter describes how the classification models were set up. The aim of every model is to predict whether a patient has heart disease (class 1) or not (class 0). Four algorithms were used: Gaussian Naive Bayes, Decision Tree, K-Nearest Neighbours and Logistic Regression. KNN was run with three different values of k, so six model runs were compared in total. All of them come from scikit-learn.

6.2 Feature Selection

All 13 features were used as input and target is the label. The numeric and categorical split is the one shown in Table 3.3. I did not use a separate feature-selection method because the number of features is small and each of them has a clinical meaning.

6.3 Feature Extraction and Transformation

The transformation is the ColumnTransformer described in Section 3.7. In short, the six numeric columns pass through median imputation and standard scaling, and the seven categorical columns pass through most-frequent imputation and one-hot encoding. After one-hot encoding, the model receives more than 13 input columns, because each category becomes its own column. The same transformer is created fresh for every model, so no fitted values are shared between models.

6.4 Training and Testing Datasets

Setting	Value
Split ratio	80% training, 20% testing
Training patients	241
Test patients	61 (33 without disease, 28 with disease)
Random state	123
Stratified	Yes, on the target
Cross-validation	5-fold stratified, shuffled, random state 123

6.5 Classification Algorithms

6.5.1 Gaussian Naive Bayes

Naive Bayes applies Bayes' theorem and assumes that the features are independent of each other given the class. The Gaussian version models each feature as a normal distribution inside every class. It is very fast and works well on small datasets, even though the independence assumption is not exactly true here. I used the default settings, GaussianNB().

6.5.2 Decision Tree

A Decision Tree learns a set of if-then rules by repeatedly splitting the data on the feature that separates the classes best. I used the entropy criterion with max_depth = 3 and random_state = 123. A shallow depth was chosen on purpose. A deep tree can memorise 241 training patients and then fail on new ones, whereas a depth of three keeps the rules short and easy to read. The fitted tree is shown in Figure 6.1.

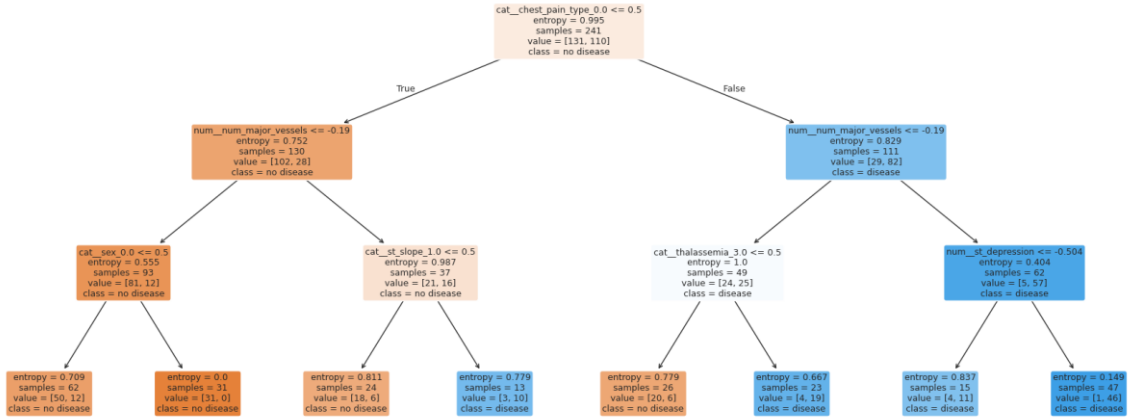


Figure 6.1: Decision tree (maximum depth 3)

6.5.3 K-Nearest Neighbours (KNN)

KNN classifies a patient by looking at the k most similar patients in the training data and taking the majority class among them. Because it depends on distances, scaling is essential. I tested k = 4, k = 7 and the value chosen by cross-validation, k = 17. Figure 6.2 shows how cross-validated accuracy on the training data changes with k.

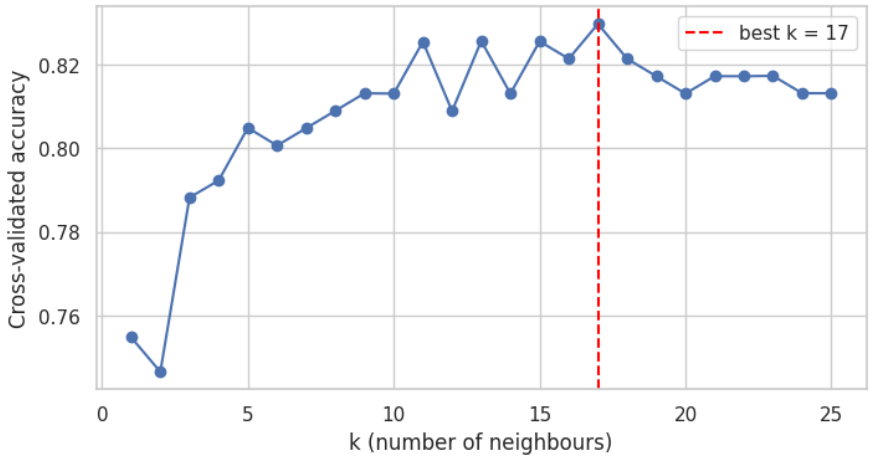


Figure 6.2: Cross-validated accuracy for k = 1 to 25 (best k = 17)

6.5.4 Logistic Regression

Logistic Regression estimates the probability of disease as a weighted combination of the input features passed through a sigmoid function. It is a simple linear model and it works well when the preprocessing is done properly. I used LogisticRegression(max_iter = 1000), where the larger iteration limit makes sure that the solver converges.

A note on a baseline

The notebook does not train a separate dummy classifier. As a rough reference, a model that always predicts “no disease” would be right for 54.3% of the patients, because that is the share of the larger class. Any useful model should clearly beat this figure.

6.6 Classifier Training and Predictions

For each classifier, a pipeline is created with make_model(), fitted on the training set and used to predict the labels and the probabilities of the test set. A common function, evaluate_model(), then calculates the confusion matrix, test accuracy, false negative rate, ROC AUC and the five-fold cross-validated accuracy, and it draws the confusion matrix.

6.7 Parameter Setup

Table 6.1: Parameters used for each model

Model	Parameters
Gaussian Naive Bayes	Default settings
Decision Tree	criterion = entropy, max_depth = 3, random_state = 123
KNN (k = 4)	n_neighbors = 4
KNN (k = 7)	n_neighbors = 7
KNN (k = 17)	n_neighbors = 17 (chosen by cross-validation on training data)
Logistic Regression	max_iter = 1000, other settings default

Apart from the choice of k, the parameters were set by hand. I did not use GridSearchCV or RandomizedSearchCV, so the models may not be at their best possible setting. This is listed again as a future improvement.

7. RESULTS AND MODEL COMPARISON

7.1 Introduction

In this chapter the six model runs are compared on the 61-patient test set and with five-fold cross-validation. I also look at the confusion matrices, the classification report of the best model and the ROC curves.

7.2 Evaluation Metrics

- **Accuracy:** the share of all predictions that are correct.
- **Precision:** out of the patients predicted to have a class, the share that really have it.
- **Recall:** out of the patients who really have a class, the share the model finds. For the disease class, recall is 100% minus the false negative rate.
- **F1-score:** the harmonic mean of precision and recall.
- **False negative rate (FNR):** the share of sick patients predicted as healthy. This is the error that matters most in a medical screening setting.
- **Cross-validated accuracy:** the mean accuracy over five folds, given with its standard deviation.
- **ROC AUC:** the probability that the model gives a higher score to a randomly chosen sick patient than to a randomly chosen healthy one. A value of 0.5 means random guessing and 1.0 means perfect ranking.

7.3 Model Performance Comparison

Table 7.1 lists the results of all models, sorted by cross-validated accuracy. Figure 7.1 shows the cross-validated accuracies as a bar chart.

Table 7.1: Performance comparison of the classification models

Model	Test accuracy %	FNR %	CV accuracy %	CV std %	ROC AUC
Logistic Regression	88.52	17.86	85.77	3.23	0.953
KNN (k = 7)	88.52	21.43	82.79	4.23	0.938
KNN (k = 17)	88.52	21.43	82.44	3.60	0.943
Naive Bayes	88.52	21.43	82.44	4.43	0.951
Decision Tree	88.52	17.86	80.50	5.26	0.909
KNN (k = 4)	86.89	25.00	80.48	2.75	0.920

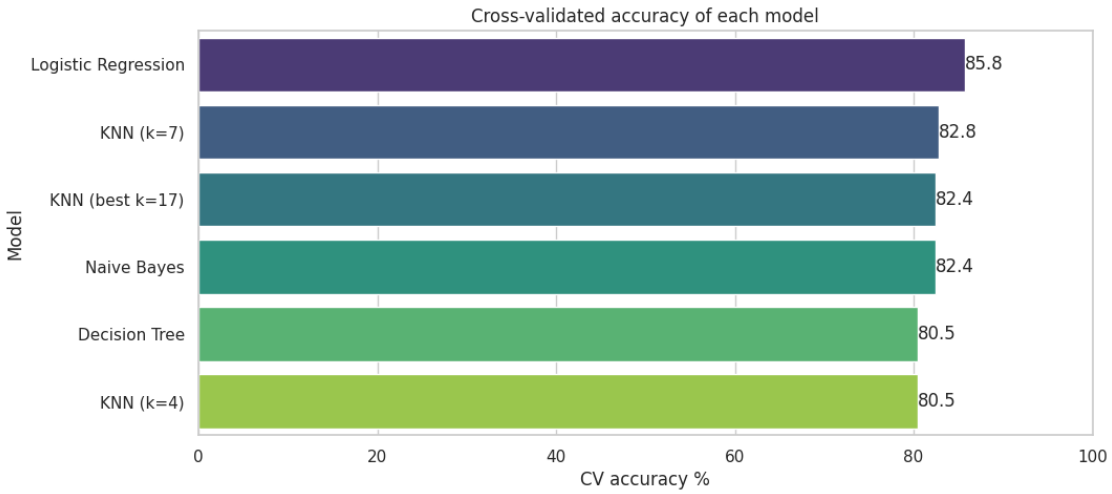


Figure 7.1: Cross-validated accuracy of each model

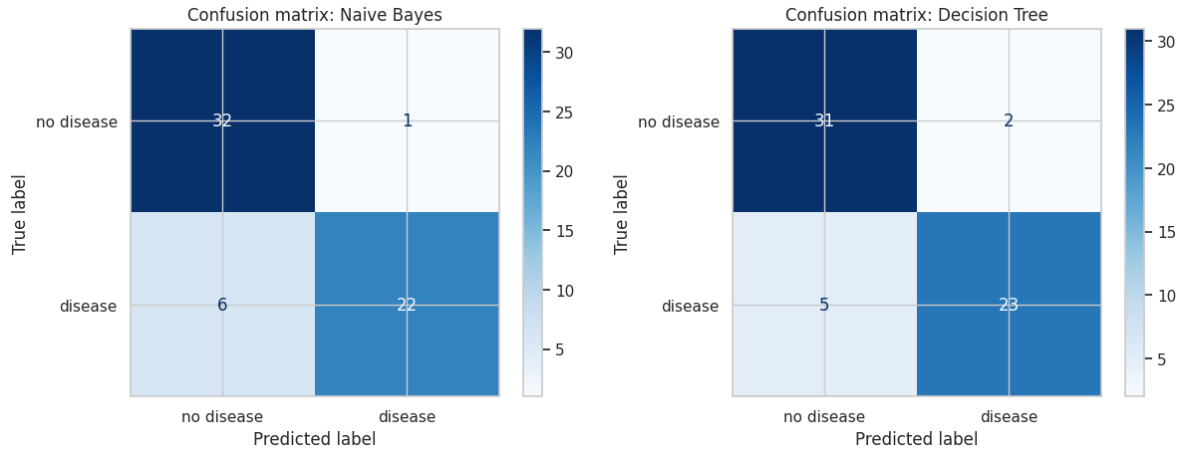
Five of the six runs reached the same test accuracy of 88.52%, which equals 54 correct predictions out of 61. Only KNN with k = 4 was slightly lower with 86.89% (53 correct). Since one patient is worth about 1.6 percentage points, such small gaps do not show that one model is truly better than another. The cross-validated scores are more useful for this reason, and there Logistic Regression is ahead with 85.77%, followed by KNN with k = 7 at 82.79%. The test accuracies are also a little higher than the cross-validated ones, which suggests this particular test split was slightly easy.

7.4 Confusion Matrix Analysis

A confusion matrix shows, for each class, how many patients were predicted correctly and how many were not. In the matrices of Figure 7.2 the “disease” class is the positive one. The counts are summarised in Table 7.2.

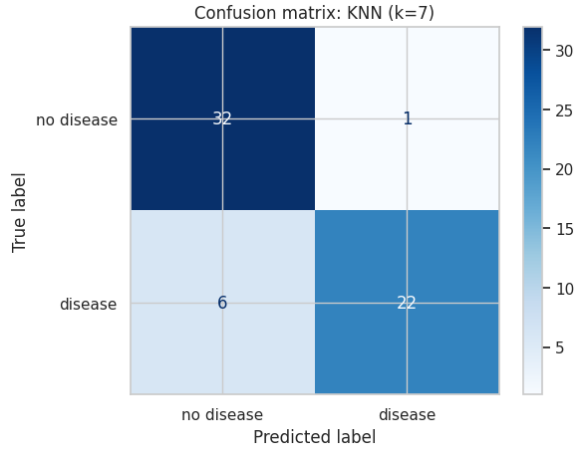
Table 7.2: Confusion matrix counts on the test set (33 healthy, 28 diseased patients)

Model	True negative	False positive	False negative	True positive
Naive Bayes	32	1	6	22
Decision Tree	31	2	5	23
KNN (k = 4)	32	1	7	21
KNN (k = 7)	32	1	6	22
KNN (k = 17)	32	1	6	22
Logistic Regression	31	2	5	23

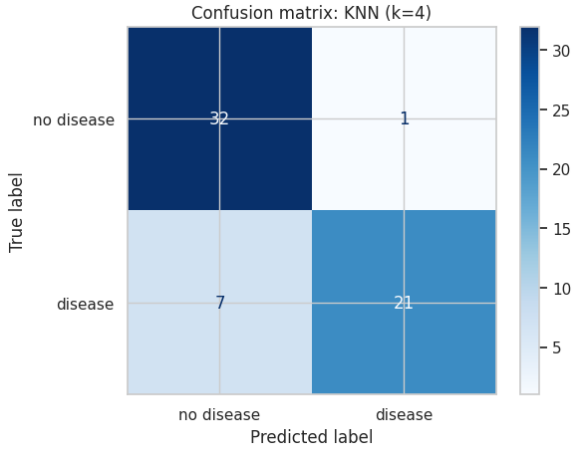


Naive Bayes

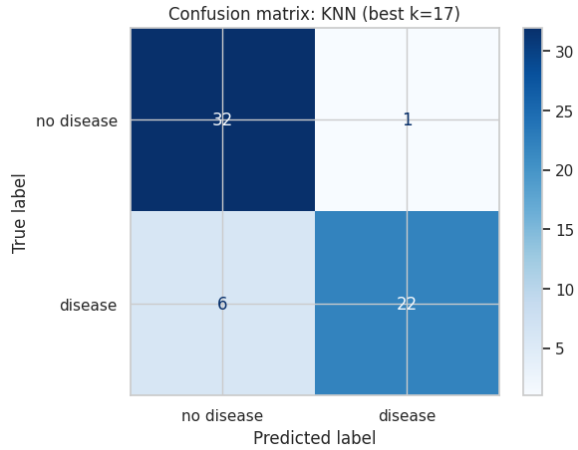
Decision Tree



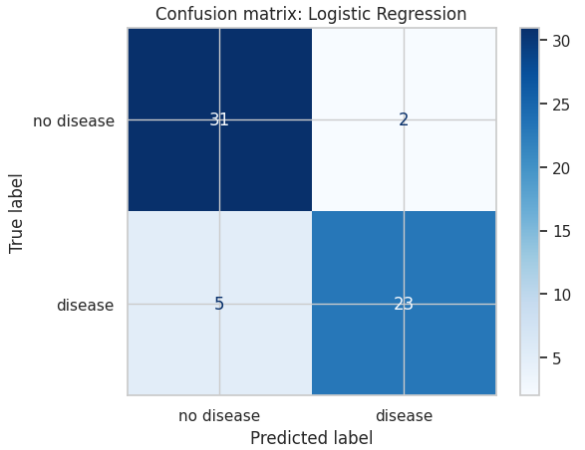
KNN (k = 7)



KNN (k = 4)



KNN (k = 17)



Logistic Regression

Figure 7.2: Confusion matrices of the models on the test set

Most of the errors are false negatives, not false positives. For example, Logistic Regression wrongly labels only 2 of the 33 healthy patients as sick, but misses 5 of the 28 sick patients. The same pattern appears for every model, with the number of missed patients ranging from 5

to 7. In a real screening situation this would be a concern, because the cost of a missed patient is higher than the cost of an extra test.

7.5 Classification Report

Table 7.3 shows the classification report of Logistic Regression, the model that scored best in cross-validation.

Table 7.3: Classification report of Logistic Regression (test set)

Class	Precision	Recall	F1-score	Support
No disease	0.86	0.94	0.90	33
Disease	0.92	0.82	0.87	28
Accuracy			0.89	61
Macro average	0.89	0.88	0.88	61
Weighted average	0.89	0.89	0.88	61

When the model says a patient has heart disease it is right 92% of the time, but it finds only 82% of the patients who are actually sick. For the healthy class the situation is the opposite: recall is high (0.94) and precision is lower (0.86). If a hospital wanted to catch more sick patients, the decision threshold could be lowered from 0.5, which would raise recall at the cost of more false alarms.

The ROC curves of all models are drawn in Figure 7.3. All curves stay well above the diagonal line of random guessing. Logistic Regression has the highest AUC (0.953), Naive Bayes follows closely (0.951), and the Decision Tree has the lowest (0.909).

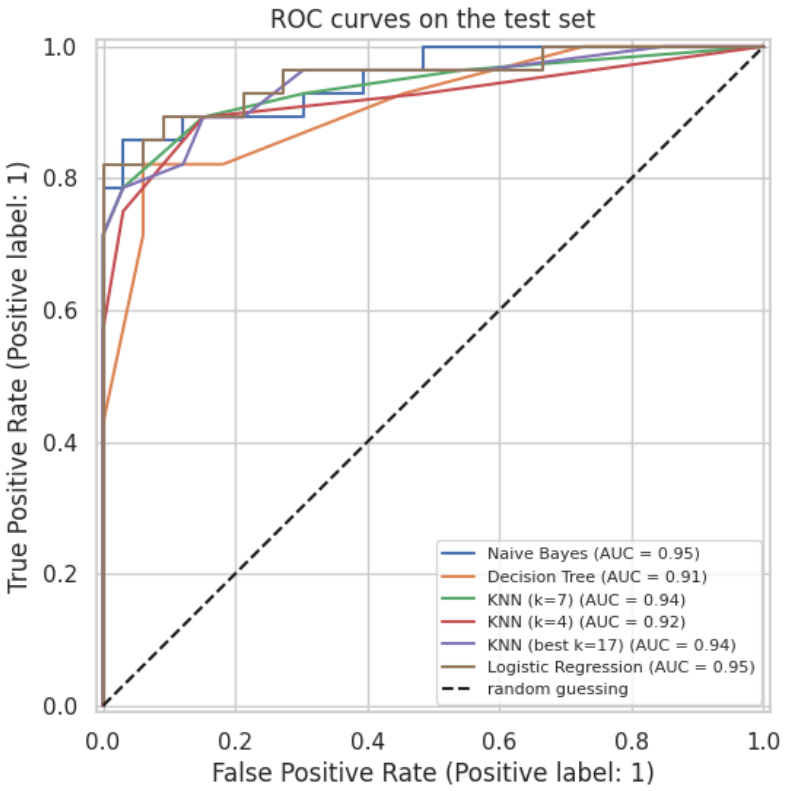


Figure 7.3: ROC curves of the models on the test set

7.6 Overall Result Analysis

Putting the results together, Logistic Regression is the best model in this project. It has the highest cross-validated accuracy (85.77%) with a small spread (3.23), the highest ROC AUC (0.953) and the joint-lowest false negative rate (17.86%, shared with the Decision Tree). The Decision Tree has the same test accuracy, but its cross-validated score is the lowest apart from KNN with k = 4 and its spread is the largest (5.26), so it is the less stable of the two. Naive Bayes and KNN perform at a similar level to each other. Choosing k = 17 by cross-validation gave a slightly better AUC than k = 7, but not a better accuracy, so tuning k brought only a small gain.

7.7 Prediction for a New Patient

To test the trained pipeline, I entered a made-up record: a 58-year-old male with asymptomatic chest pain, resting blood pressure of 140, cholesterol of 240, maximum heart rate of 120, exercise-induced angina, ST depression of 2.0, a flat ST slope, one major vessel and a reversible thalassemia defect. Logistic Regression gave a probability of heart disease of 98.3% and therefore the label “disease”. This record is invented and only shows how the model is used. It is not a medical result.

8. CONCLUSION AND LIMITATIONS

8.1 Conclusion

In this project I built a complete machine learning workflow for predicting heart disease from the Cleveland dataset. The raw file looked clean at first, but a closer look showed one duplicate row, seven impossible category codes and a target column whose meaning was reversed compared with the natural reading. Correcting these problems and placing all preprocessing inside a pipeline made the evaluation trustworthy.

The exploratory analysis showed that male sex, older age, a low maximum heart rate, exercise-induced angina, higher ST depression and a larger number of affected vessels go together with heart disease. Among the models tested, Logistic Regression performed best, with 88.52% test accuracy, 85.77% cross-validated accuracy and a ROC AUC of 0.953. The other models were close, and the differences between them on a 61-patient test set are small. Even the best model still misses about 18% of the sick patients, so the results show a good learning exercise, not a ready diagnostic tool.

8.2 Limitations

- **Small dataset:** only 302 patients were used, all from one hospital. A model trained on this data may not behave the same way for patients from other places or age groups.
- **Small test set:** the 61 test patients make the test accuracy unstable. This is why cross-validation was added, but even that has a spread of 3 to 5 percentage points.
- **False negatives:** the false negative rate of the best models is around 18%, which would be too high for a real screening setting.
- **Binary target:** the original database records the severity of the disease, but here it is reduced to presence or absence.
- **Label direction:** the decision to flip the target was based on patterns in the data and not on written documentation, so the dataset description should be checked if the work is extended.
- **Limited tuning:** apart from k in KNN, all parameters were chosen by hand and no systematic search was run.

8.3 Future Enhancements

- Add the Hungarian, Swiss and Long Beach parts of the UCI collection, or another dataset, to test whether the model works on other patient groups.

- Try Random Forest, Support Vector Machine and gradient boosting models.
- Use GridSearchCV or RandomizedSearchCV to tune the hyperparameters.
- Lower the decision threshold, or use class weights, to reduce the false negative rate, and check the calibration of the predicted probabilities.
- Study feature importance, for example the coefficients of the logistic model, to explain which measurements drive a prediction.
- Build a small web interface where a user can enter values and see the predicted risk.

8.4 Final Remarks

The project helped me to understand that a good machine learning result depends as much on careful data checking and fair evaluation as on the choice of the algorithm. A simple model with correct preprocessing did better than more flexible ones, and the small differences between models were only visible after cross-validation. I hope the report shows these steps clearly.

9.CODE

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix, ConfusionMatrixDisplay,
                             classification_report, roc_auc_score, RocCurveDisplay)

RANDOM_STATE = 123            # fixed seed -> same results every run
np.random.seed(RANDOM_STATE)
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)   # show all columns in tables
pd.set_option("display.width", 200)

print("pandas", pd.__version__, "| numpy", np.__version__)
```

```
data = pd.read_csv("heart.csv")
print("Loaded heart.csv ->", data.shape[0], "rows x", data.shape[1], "columns")
data.head()
data.sample(5, random_state=RANDOM_STATE)
data.describe()
data.info()
print("Missing values (NaN):", int(data.isnull().sum().sum()))
print("Duplicate rows      :", int(data.duplicated().sum()))
print("Missing values (NaN):", int(data.isnull().sum().sum()))
print("Duplicate rows      :", int(data.duplicated().sum()))
data = data.drop_duplicates().reset_index(drop=True)
data.loc[data["ca"] == 4, "ca"] = np.nan
data.loc[data["thal"] == 0, "thal"] = np.nan
print("Rows after removing duplicates:", len(data))
print("Missing values now:\n", data.isnull().sum()[lambda s: s > 0])
print("Share with target = 1 :", round(data["target"].mean() * 100, 1), "%")
print(data.groupby("sex")["target"].mean().rename({0: "female", 1: "male"}).round(2))
FLIP_TARGET = True
if FLIP_TARGET:
    data["target"] = 1 - data["target"]
```

```
print(data["target"].value_counts().rename({0: "no disease (0)", 1: "disease (1)"}))
data.columns = ["age", "sex", "chest_pain_type", "resting_blood_pressure", "cholesterol",
                "fasting_blood_sugar", "rest_ecg", "max_heart_rate_achieved",
                "exercise_induced_angina", "st_depression", "st_slope",
                "num_major_vessels", "thalassemia", "target"]

label_maps = {
    "sex":                    {0: "female", 1: "male"},
    "chest_pain_type":        {0: "asymptomatic", 1: "atypical angina", 2: "non-anginal pain", 3: "typical angina"},
    "fasting_blood_sugar":    {0: "<= 120 mg/dl", 1: "> 120 mg/dl"},
    "rest_ecg":               {0: "left ventricular hypertrophy", 1: "normal", 2: "ST-T wave abnormality"},
    "exercise_induced_angina": {0: "no", 1: "yes"},
    "st_slope":               {0: "downsloping", 1: "flat", 2: "upsloping"},
    "thalassemia":            {1: "fixed defect", 2: "normal", 3: "reversible defect"},
    "target":                 {0: "no disease", 1: "disease"},
}

eda = data.copy()
for column, mapping in label_maps.items():
    eda[column] = eda[column].map(mapping)       # .map() is the safe way to replace codes

eda.head()
ax = sns.countplot(x="target", data=eda, order=["no disease", "disease"], hue="target",
                   palette="bwr", legend=False)
ax.set_title("Patients with and without heart disease")
plt.show()

print(eda["target"].value_counts())
pct_female = (data["sex"] == 0).mean() * 100
pct_male   = (data["sex"] == 1).mean() * 100
print(f"Percentage of female patients: {pct_female:.2f}%")
print(f"Percentage of male patients  : {pct_male:.2f}%")
```

```
pct_no  = (data["target"] == 0).mean() * 100
pct_yes = (data["target"] == 1).mean() * 100
print(f"Percentage without heart disease: {pct_no:.2f}%")
print(f"Percentage with heart disease   : {pct_yes:.2f}%")
sns.barplot(x="sex", y="target", data=data)
plt.xticks([0, 1], ["female", "male"])
plt.ylabel("share with heart disease")
plt.title("Heart disease rate by sex")
plt.show()
data.groupby('target').mean(numeric_only=True).round(2)
pd.crosstab(eda["age"], eda["target"]).plot(kind="bar", figsize=(20, 6))
plt.title("Heart disease frequency for ages")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.savefig("heartDiseaseAndAges.png")
plt.show()

pd.crosstab(eda["sex"], eda["target"]).plot(kind="bar", figsize=(10, 5), color=["blue", "#AA1111"])
plt.title("Heart disease frequency for sex")
plt.xlabel("Sex")
plt.xticks(rotation=0)
plt.legend(["No disease", "Disease"])
plt.ylabel("Frequency")
plt.show()
plt.figure(figsize=(8, 6))
sns.scatterplot(x="cholesterol", y="thalassemia", data=eda.dropna(subset=["thalassemia"]), hue="target")
plt.show()
plt.figure(figsize=(8, 6))
sns.scatterplot(x="thalassemia", y="resting_blood_pressure", data=eda.dropna(subset=["thalassemia"]), hue="target")
plt.show()
sick    = data[data["target"] == 1]
healthy = data[data["target"] == 0]
plt.scatter(sick["age"], sick["max_heart_rate_achieved"], c="green", label="Disease")
plt.scatter(healthy["age"], healthy["max_heart_rate_achieved"], label="No disease")
```

```
plt.legend()
plt.xlabel("Age")
plt.ylabel("Maximum heart rate")
plt.show()
pd.crosstab(eda["fasting_blood_sugar"], eda["target"]).plot(kind="bar", figsize=(8, 5), color=["#4286f4", "#f49242"])
plt.title("Heart disease according to fasting blood sugar")
plt.xlabel("Fasting blood sugar")
plt.xticks(rotation=0)
plt.legend(["No disease", "Disease"])
plt.ylabel("Number of patients")
plt.show()
data.isnull().sum()
names = ["age", "resting_blood_pressure", "cholesterol", "max_heart_rate_achieved",
         "st_depression", "num_major_vessels"]
df_corr = data.loc[:, names]
corr = df_corr.corr()

fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr, annot=True, cmap="coolwarm", linewidths=0.1, ax=ax)
plt.show()
X = data.drop("target", axis=1)
y = data["target"].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y)

print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("y_train:", y_train.shape, "| y_test:", y_test.shape)
numeric_cols = ["age", "resting_blood_pressure", "cholesterol", "max_heart_rate_achieved",
                "st_depression", "num_major_vessels"]
category_cols = ["sex", "chest_pain_type", "fasting_blood_sugar", "rest_ecg",
                 "exercise_induced_angina", "st_slope", "thalassemia"]
```

```
def make_preprocessor():
    return ColumnTransformer([
        ("num", Pipeline([("fill", SimpleImputer(strategy="median")),
                          ("scale", StandardScaler())]), numeric_cols),
        ("cat", Pipeline([("fill", SimpleImputer(strategy="most_frequent")),
                          ("onehot", OneHotEncoder(handle_unknown="ignore"))]), category_cols),
    ])

def make_model(classifier):
    return Pipeline([("prep", make_preprocessor()), ("model", classifier)])

results = []
fitted_models = {}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

def evaluate_model(name, classifier):
    pipe = make_model(classifier)
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).ravel()
    accuracy = accuracy_score(y_test, y_pred) * 100
    fnr = fn * 100 / (fn + tp)
    cv_scores = cross_val_score(make_model(classifier), X, y, cv=cv, scoring="accuracy")
    auc = roc_auc_score(y_test, pipe.predict_proba(X_test)[:, 1])

    results.append({"Model": name, "Test accuracy %": round(accuracy, 2), "FNR %": round(fnr, 2),
                    "CV accuracy %": round(cv_scores.mean() * 100, 2),
                    "CV std %": round(cv_scores.std() * 100, 2), "ROC AUC": round(auc, 3)})
    fitted_models[name] = pipe
```

Loading...

```
plt.figure(figsize=(8, 4))
plt.plot(k_values, k_scores, marker="o")
plt.axvline(best_k, color="red", linestyle="--", label=f"best k = {best_k}")
plt.xlabel("k (number of neighbours)")
plt.ylabel("Cross-validated accuracy")
plt.legend()
plt.show()
knn_best_pipe, y_pred_knn_best = evaluate_model(f'KNN (best k={best_k})', KNeighborsClassifier(n_neighbors=best_k))
lr_pipe, y_pred_lr = evaluate_model('Logistic Regression', LogisticRegression(max_iter=1000))
print(classification_report(y_test, y_pred_lr, target_names=['no disease', 'disease']))
results_df = pd.DataFrame(results).sort_values("CV accuracy %", ascending=False).reset_index(drop=True)
results_df
plt.figure(figsize=(11, 5))
ax = sns.barplot(x="CV accuracy %", y="Model", data=results_df, hue="Model", palette="viridis", legend=False)
ax.set_xlim(0, 100)
for container in ax.containers:
    ax.bar_label(container, fmt="%.1f")
plt.title("Cross-validated accuracy of each model")
plt.show()
fig, ax = plt.subplots(figsize=(7, 6))
for name, pipe in fitted_models.items():
    RocCurveDisplay.from_estimator(pipe, X_test, y_test, name=name, ax=ax)
ax.plot([0, 1], [0, 1], "k--", label="random guessing")
ax.legend(fontsize=8)
plt.title("ROC curves on the test set")
plt.show()
best = results_df.iloc[0]
print(f"Best model by cross-validation: {best['Model']} "
      f"({best['CV accuracy %']:.2f}% accuracy, ROC AUC on test set = {best['ROC AUC']})")
```

```
    print(f"{name}: test accuracy = {accuracy:.2f}%, FNR = {fnr:.2f}%, "
          f"5-fold CV accuracy = {cv_scores.mean() * 100:.2f}% (+/- {cv_scores.std() * 100:.2f})")
    ConfusionMatrixDisplay.from_predictions(
        y_test, y_pred, display_labels=["no disease", "disease"], cmap="Blues")
    plt.title(f"Confusion matrix: {name}")
    plt.show()
    return pipe, y_pred

nb_pipe, y_pred_nb = evaluate_model('Naive Bayes', GaussianNB())
dt_pipe, y_pred_dt = evaluate_model(
    "Decision Tree",
    DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=RANDOM_STATE))

feature_names = dt_pipe.named_steps["prep"].get_feature_names_out()
plt.figure(figsize=(22, 9))
plot_tree(dt_pipe.named_steps["model"], feature_names=feature_names,
          class_names=["no disease", "disease"], filled=True, rounded=True, fontsize=9)
plt.show()
knn7_pipe, y_pred_knn7 = evaluate_model('KNN (k=7)', KNeighborsClassifier(n_neighbors=7))
knn4_pipe, y_pred_knn4 = evaluate_model('KNN (k=4)', KNeighborsClassifier(n_neighbors=4))
k_values = list(range(1, 26))
k_scores = []
for k in k_values:
    scores_k = cross_val_score(make_model(KNeighborsClassifier(n_neighbors=k)),
                               X_train, y_train, cv=cv, scoring="accuracy")
    k_scores.append(scores_k.mean())

best_k = k_values[int(np.argmax(k_scores))]
print("Best k from cross-validation on training data:", best_k)
```

```python
new_patient = pd.DataFrame([{
    "age": 58, "sex": 1, "chest_pain_type": 0, "resting_blood_pressure": 140,
    "cholesterol": 240, "fasting_blood_sugar": 0, "rest_ecg": 1,
    "max_heart_rate_achieved": 120, "exercise_induced_angina": 1, "st_depression": 2.0,
    "st_slope": 1, "num_major_vessels": 1, "thalassemia": 3}])

probability = lr_pipe.predict_proba(new_patient)[0, 1]
print(f"Logistic Regression: probability of heart disease = {probability * 100:.1f}%")
print("Prediction:", "disease" if probability >= 0.5 else "no disease")
```

10. REFERENCES

1. Janosi, A., Steinbrunner, W., Pfisterer, M., and Detrano, R. Heart Disease Dataset, UCI Machine Learning Repository. Available at: https://archive.ics.uci.edu/dataset/45/heart+disease
2. Detrano, R., Janosi, A., Steinbrunner, W., et al. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. The American Journal of Cardiology, 64(5), 304–310.
3. Heart Disease (Cleveland) dataset, file heart.csv, as shared on Kaggle https://www.kaggle.com/datasets/ritwikb3/heart-disease-cleveland
4. Project source code: Heart Disease Prediction notebook. GitHub repository: https://github.com/Ash19-23/Heart_disease_prediction.git
5. LinkedIn link: https://lnkd.in/p/gizfnWc4